

WebDriverManager

Boni García

Version 6.4.0, 23-09-2026

Table of Contents

1. Motivation	1
1.1. WebDriverManager and Selenium Manager	2
2. Setup	3
3. Features	4
3.1. Driver Management	4
3.2. Browser Finder	8
3.3. WebDriver Builder	9
3.4. Browsers in Docker	11
3.5. Browsers Monitoring	17
4. Other Usages	24
4.1. WebDriverManager CLI	24
4.2. WebDriverManager Server	26
4.3. WebDriverManager Agent	27
4.4. Selenium-Jupiter	28
4.5. Selenium Grid	29
4.6. Appium	30
5. Examples	31
6. Advanced Configuration	32
7. Known Issues	40
7.1. HTTP response code 403	40
7.2. Testing localhost in Docker	40
7.3. Chrome 92-94 in Docker	41
7.4. Support for chromedriver 115+	41
7.5. Connectivity issues	42
8. Troubleshooting	43
9. Community	46
10. Support	47
11. Further Documentation	48
12. About	49

Chapter 1. Motivation

[Selenium WebDriver](#) is a library that allows controlling web browsers programmatically. It provides a cross-browser API that can be used to drive web browsers (e.g., Chrome, Edge, or Firefox, among others) using different programming languages (e.g., Java, JavaScript, Python, C#, or Ruby). The primary use of Selenium WebDriver is implementing automated tests for web applications.

Selenium WebDriver carries out the automation using the native support of each browser. For this reason, we need to place a binary file called *driver* between the test using the Selenium WebDriver API and the browser to be controlled. Examples of drivers for major web browsers nowadays are [chromedriver](#) (for Chrome), [geckodriver](#) (for Firefox), or [msedgedriver](#) (for Edge). As you can see in the following picture, the communication between the WebDriver API and the driver binary is done using a standard protocol called [W3C WebDriver](#) (formerly the so-called *JSON Wire Protocol*). Then, the communication between the driver and the browser is done using the native capabilities of each browser.

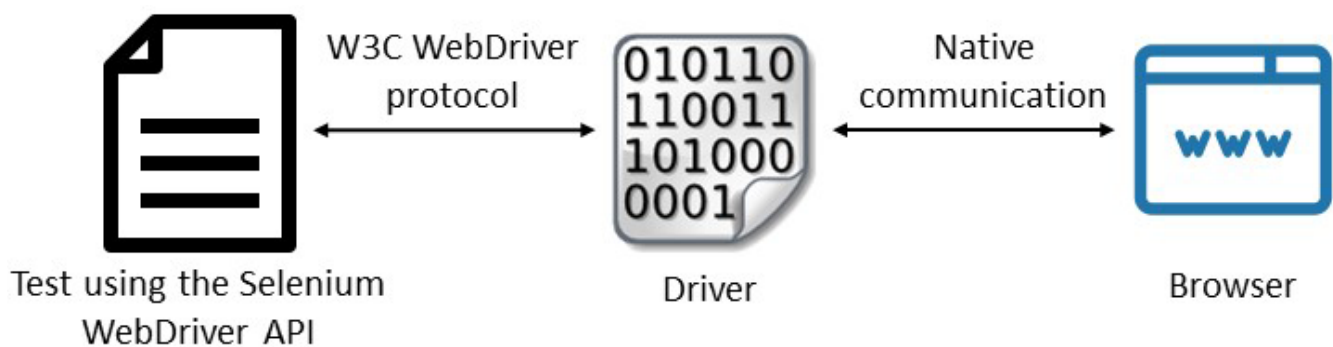


Figure 1. Selenium WebDriver Architecture

From a practical point of view, we need to make a *driver management process* to use Selenium WebDriver. This process consists on:

1. Download. Drivers are platform-specific binary files. To download the proper driver, we have to identify the driver type we need (e.g., chromedriver if we want to use Chrome), the operating system (typically, Windows, Linux, or Mac OS), the architecture (typically, 32 or 64 bits), and very important, the driver version. Concerning the version, each driver release is usually compatible with a given browser version(s). For this reason, we need to discover the correct driver version for a specific browser release (typically reading the driver documentation or release notes).
2. Setup. Once we have downloaded the driver to our computer, we need to provide a way to locate this driver from our Selenium WebDriver tests. In Java, this setup can be done in two different ways. First, we can add the driver location to our `PATH` environmental variable. Second, we can use *Java system properties* to export the driver path. Each driver path should be identified using a given system property, as follows:

```
System.setProperty("webdriver.chrome.driver", "/path/to/chromedriver");  
System.setProperty("webdriver.gecko.driver", "/path/to/geckodriver");  
System.setProperty("webdriver.edge.driver", "/path/to/msedgedriver");  
System.setProperty("webdriver.opera.driver", "/path/to/operadriver");
```

```
System.setProperty("webdriver.ie.driver", "C:/path/to/IEDriverServer.exe");
```

3. Maintenance. Last but not least, we need to warranty the compatibility between driver and browser in time. This step is relevant since modern browsers automatically upgrade themselves (i.e., they are *evergreen* browsers), and for this reason, the compatibility driver-browser is not warranted in the long run. For instance, when a WebDriver test using Chrome faces a driver incompatibility, it reports the following error message: *"this version of chromedriver only supports chrome version N."* As you can see in [StackOverflow](#), this is a recurrent problem for manually managed drivers (chromedriver in this case).

What is WebDriverManager?

[WebDriverManager](#) is an open-source Java library that carries out the management (i.e., download, setup, and maintenance) of the drivers required by Selenium WebDriver (e.g., chromedriver, geckodriver, msedgedriver, etc.) in a fully automated manner. In addition, as of version 5, WebDriverManager provides other relevant features, such as the capability to [discover browsers](#) installed in the local system, [building WebDriver objects](#) (such as [ChromeDriver](#), [FirefoxDriver](#), [EdgeDriver](#), etc.), running [browsers in Docker containers](#) seamlessly, and [monitoring capabilities](#).

1.1. WebDriverManager and Selenium Manager

Maybe you have heard (and if not, you should) about [Selenium Manager](#). Selenium Manager is the official driver manager of the Selenium project, and it is shipped out of the box with every Selenium release. You might have some doubts about this:

Is Selenium Manager a replacement for WebDriverManager? For the use case of automated driver management, yes. In other words, if you use WebDriverManager only for driver management, you can safely switch to Selenium Manager.

What are the differences between WebDriverManager and Selenium Manager? Both projects provide automated driver management (for chromedriver, geckodriver, etc.). But, WebDriverManager provides several features not available in Selenium Manager (e.g., self-managed browsers in Docker containers or custom monitoring features). On the other side, Selenium Manager provides automated browser management (e.g., based on [Chrome for Testing](#)).

Then, should I move to Selenium Manager? It depends. If you use some custom feature of WebDriverManager, you can continue using it. If you use WebDriverManager only for automated management, you can switch to Selenium Manager. However, if you cannot bump to Java 11 (which is the [minimum Java version for the latest versions of Selenium by September 2023](#)), WebDriverManager can still be your library for driver management, since WebDriverManager will continue supporting Java 8 (at least for some time more).

Will the WebDriverManager development stop? WebDriverManager might still be helpful, so its development and maintenance continue.

Chapter 2. Setup

WebDriverManager is primarily used as a Java dependency (although [other usages](#) are also possible). We typically use a *build tool* (such as [Maven](#) or [Gradle](#)) to resolve the WebDriverManager dependency. In Maven, it can be done as follows (notice that it is declared using the `test` scope, since it is typically used in tests classes):

```
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>6.4.0</version>
  <scope>test</scope>
</dependency>
```

In the case of a Gradle project, we can declare WebDriverManager as follows (again, for tests):

```
dependencies {
    testImplementation("io.github.bonigarcia:webdrivermanager:6.4.0")
}
```

Chapter 3. Features

WebDriverManager provides a fluent API available using the class `WebDriverManager` (package `io.github.bonigarcia.wdm`). This class provides a group of static methods to create *managers*, i.e., objects devoted to providing automated driver management and other features.

3.1. Driver Management

The primary use of WebDriverManager is the automation of driver management. To use this feature, you need to select a given manager in the WebDriverMager API (e.g., `chromedriver()` for Chrome) and invoke the method `setup()`. The following example shows a test case using `JUnit 5`, Selenium WebDriver, WebDriverManager, and `AssertJ` (for fluent assertions). In this test, we invoke WebDriverManager in the setup method for all tests (`@BeforeAll`). This way, the required driver (chromedriver) will be available for all the WebDriver tests using Chrome in this class.

```
import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class ChromeTest {

    WebDriver driver;

    @BeforeAll
    static void setupClass() {
        WebDriverManager.chromedriver().setup();
    }

    @BeforeEach
    void setupTest() {
        driver = new ChromeDriver();
    }

    @AfterEach
    void teardown() {
        driver.quit();
    }

    @Test
    void test() {
        // Exercise
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
    }
}
```

```

String title = driver.getTitle();

// Verify
assertThat(title).contains("Selenium WebDriver");
}

}

```

WebDriverManager provides a set of *managers* for Chrome, Firefox, Edge, Opera, Chromium, and Internet Explorer. The basic use of these managers is the following:

```

WebDriverManager.chromedriver().setup();
WebDriverManager.firefoxdriver().setup();
WebDriverManager.edgedriver().setup();
WebDriverManager.operadriver().setup();
WebDriverManager.chromiumdriver().setup();
WebDriverManager.iedriver().setup();

```

As of version 5, WebDriverManager also provides a manager for Safari (called `safaridriver()`). The case of the Safari browser is particular since this browser does not require to manage its driver to work with Selenium WebDriver (in other words, the Safari driver is built-in within the browser). Nevertheless, WebDriverManager provides this manager to be used in the WebDriver builder (especially with [Docker](#)).



Although not mandatory, it is highly recommended to use a logger library to trace your application and tests. In the case of WebDriverManager, you will see the relevant steps of the driver management following its traces. See for example the following [tutorial](#) to use [SLF4J](#) and [Logback](#). Also, you can see an example of a WebDriverManager test using logging [here](#) (this example uses this [configuration file](#)).

3.1.1. Resolution Algorithm

WebDriverManager executes a *resolution algorithm* when calling to `setup()` in a given manager. You can find all its internal details in the paper [Automated driver management for Selenium WebDriver](#), published in the Springer Journal of Empirical Software Engineering in 2021. The most relevant parts of this algorithm are the following:

1. WebDriverManager tries to find the browser version. To this aim, WebDriverManager uses internally a knowledge database called [commands database](#). This database is a collection of shell commands used to discover the version of a given browser in the different operating systems (e.g., `google-chrome --version` for Chrome in Linux). As of WebDriverManager 6.0.0, you can specify the browser binary path (using the method `browserBinary()`) that WebDriverManager will use for browser version discovery.
2. Using the browser version, WebDriverManager tries to find the proper driver version. This process is different for each browser. For chromedriver, the [Chrome for Testing \(CfT\) endpoints](#) are used. For geckodriver, the [Firefox mapping maintained by the Selenium project](#) (which is

based on the [official geckodriver support](#)). For msedgedriver, the [Edge metadata](#) is used.

3. Once the driver version is discovered, WebDriverManager downloads this driver to a local cache (located at `~/.cache/selenium` by default). These drivers are reused in subsequent calls.
4. Finally, WebDriverManager exports the driver path using Java system properties (e.g., `webdriver.chrome.driver` in the case of the Chrome manager).

This process automated the first two stages of the driver management previously introduced, i.e., download and setup. To support the third stage (i.e., maintenance), WebDriverManager implements *resolution cache*. This cache (called by default `resolution.properties` and stored in the root of the driver cache) is a file that stores the relationship between the resolved driver and browser versions. This relationship is valid during a given *time-to-live* (TTL). The default value for this TTL is 1 hour for browsers and 1 day for drivers. In other words, the discovered browser version is valid for 1 hour, and the driver version is considered correct for 1 day. This mechanism improves the performance dramatically since the second (and following) calls to the resolution algorithm for the same browser are resolved using only local resources (i.e., without using the shell nor requesting external services).

3.1.2. Generic Manager

WebDriverManager provides a *generic manager*, i.e., a manager that can be parameterized to act as a specific manager (for Chrome, Firefox, etc.). Using this feature, you can create a manager using the method `getInstance()` of the WebDriverManager API. The method can be invoked using the following options:

- `getInstance(Class<? extends WebDriver> webDriverClass)`: Where `webDriverClass` is a class of the Selenium WebDriver standard hierarchy, such as `ChromeDriver.class`, `FirefoxDriver.class`, etc.
- `getInstance(DriverManagerType driverManagerType)`: Where `driverManagerType` is an [enumeration](#) provided by WebDriverManager to identify the available managers.
- `getInstance(String browserName)`: Where `browserName` is the usual browser name as `String` (i.e., "Chrome", "Firefox", "Edge", "Opera", "Chromium", "Safari", or "IExplorer").
- `getInstance()`: If no parameter is specified, the configuration key `wdm.defaultBrowser` is used to select the manager (Chrome by default). See the [advanced configuration](#) section for further information about the configuration capabilities of WebDriverManager.

The following example shows a [JUnit 5 parameterized test](#) in which the test is repeated twice (using the classes `ChromeDriver.class` and `FirefoxDriver.class` as test parameters). As you can see, WebDriverManager uses this parameter to instantiate the proper manager and create the `WebDriver` instance (see [WebDriver Builder](#) section for more information about this feature).

```
import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
```

```

import io.github.bonigarcia.wdm.WebDriverManager;

class GenericTest {

    WebDriver driver;

    @AfterEach
    void teardown() {
        driver.quit();
    }

    @ParameterizedTest
    @ValueSource(classes = { ChromeDriver.class, FirefoxDriver.class })
    void test(Class<? extends WebDriver> webDriverClass) {
        // Driver management and WebDriver instantiation
        driver = WebDriverManager.getInstance(webDriverClass).create();

        // Exercise
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        String title = driver.getTitle();

        // Verify
        assertThat(title).contains("Selenium WebDriver");
    }
}

```

3.1.3. Advanced Settings

The driver management provided by `WebDriverManager` can be tuned in many different ways. For example, the `WebDriverManager` API allows configuring the driver (or browser) version to be resolved, the location of the cache path, the operating system, the architecture, the proxy settings (for the network connection), the TTL, or cleaning the driver and resolution cache, among many other attributes. See the [advanced configuration](#) settings for specific details about it. In addition, there are several `WebDriverManager` API methods specific to driver management, namely:

- `getDownloadedDriverPath()`: Used to find out the path of the resolved driver in the current instance of `WebDriverManager`.
- `getDownloadedDriverVersion()`: Used to find out the version of the driver resolved in the current instance of `WebDriverManager`.
- `getResolvedBrowserVersion()`: Used to find out the major browser version discovered in the current instance of `WebDriverManager`.
- `getDriverVersions()`: Used to find the list of available driver versions in a given manager.
- `getDriverManagerType()`: Used to get the driver manager type (and `enum`) of a given manager.



Each manager was a singleton object in older `WebDriverManager` releases (e.g.,

4.x), while in version 5, a new manager instance is created each time. Therefore, the usage of `getDownloadedDriverPath()` and `getDownloadedDriverVersion()` can be different in WebDriverManager 5 (i.e., these methods need to be invoked using a WebDriverManager instance previously created).

3.2. Browser Finder

As of version 5, WebDriverManager allows detecting if a given browser is installed or not in the local system. To this aim, each manager provides the method `getBrowserPath()`. This method returns an `Optional<Path>`, which is empty if a given browser is not installed in the system or the browser path (within the optional object) when detected.

The following example shows an example using this feature. In this test, the optional browser path is used to disable conditionally (i.e., skip) the test using an `AssertJ assumption` (although other built-in assumptions available in JUnit 5 or other unit testing frameworks are also possible). This test should be executed in a Mac OS system (which should have Safari out of the box), but it should be skipped in any other operating system.

```
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assumptions.assumeThat;

import java.nio.file.Path;
import java.util.Optional;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.safari.SafariDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class SafariTest {

    WebDriver driver;

    @BeforeAll
    static void setupClass() {
        Optional<Path> browserPath = WebDriverManager.safaridriver()
            .getBrowserPath();
        assumeThat(browserPath).isPresent();
    }

    @BeforeEach
    void setupTest() {
        driver = new SafariDriver();
    }
}
```

```

@AfterEach
void teardown() {
    driver.quit();
}

@Test
void test() {
    // Exercise
    driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
    String title = driver.getTitle();

    // Verify
    assertThat(title).contains("Selenium WebDriver");
}
}

```



Internally, WebDriverManager uses the content of the [commands database](#) to detect the possible browser paths in different operating systems.

3.3. WebDriver Builder

As of version 5, WebDriverManager allows instantiating **WebDriver** objects (e.g. **ChromeDriver**, **FirefoxDriver**, etc.) using the WebDriverManager API. This feature is available using each manager's method **create()**. The following example shows a test using this feature. Notice that the WebDriverManager call to the **setup()** method is not required when using this feature since the driver management is done internally by WebDriverManager. WebDriverManager provides the method **quit()** to close the created **WebDriver** instances gracefully to complement this feature.

```

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class ChromeCreateTest {

    WebDriver driver;

    @BeforeEach
    void setupTest() {
        driver = WebDriverManager.chromedriver().create();
    }

    @AfterEach
    void teardown() {

```

```

        driver.quit();
    }

    @Test
    void test() {
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        assertThat(driver.getTitle()).contains("Selenium WebDriver");
    }
}

```



When using this feature, WebDriverManager stores internally a reference to the **WebDriver** objects created. In addition, a shutdown hook watches these objects correctly released before shutting down the JVM. You can play with this feature by removing the **tearDown** method of the example before.

The WebDriverManager API provides different methods to enhance the creation of **WebDriver** objects, such as:

- Integer parameter in the method **create()**. This option is used to create a list of **WebDriver** objects instead of a single instance. See [example](#).
- Method **capabilities()**: To specify WebDriver **Capabilities** (see [example](#)).
- Method **remoteAddress()**: To specify the remote URL in the **RemoteWebDriver** instance, typically when using a Selenium Server or a cloud provider (such as [Sauce Labs](#), [LambdaTest](#), etc.). This method is equivalent to the configuration key **wdm.remoteAddress** (see [configuration](#) section). The following example shows a test using this method. This test starts **Selenium Grid** in standalone mode before the test. To that, WebDriverManager is also used (since the browser controlled by Selenium Grid also requires a driver):

```

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.grid.Main;

import io.github.bonigarcia.wdm.WebDriverManager;

@Disabled("Broken in Selenium 4.41")
class ChromeRemoteTest {

    WebDriver driver;

    @BeforeAll
    static void setupClass() {

```

```

// Resolve driver for Selenium Grid
WebDriverManager.chromedriver().setup();

// Start Selenium Grid in standalone mode
Main.main(new String[] { "standalone", "--port", "4445" });
}

@BeforeEach
void setupTest() {
    driver = WebDriverManager.chromedriver()
        .remoteAddress("http://localhost:4445/wd/hub").create();
}

@AfterEach
void teardown() {
    driver.quit();
}

@Test
void test() {
    driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
    assertThat(driver.getTitle()).contains("Selenium WebDriver");
}
}

```

3.4. Browsers in Docker

Another relevant new feature available in WebDriverManager 5 is the ability to create browsers in [Docker](#) containers out of the box. The requirement to use this feature is to have installed a [Docker Engine](#) in the machine running the tests. To use it, we need to invoke the method `browserInDocker()` in conjunction with `create()` of a given manager. This way, WebDriverManager pulls the image from [Docker Hub](#), starts the container, and instantiates the `WebDriver` object to use it. The following test shows a simple example using Chrome in Docker:

```

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class DockerChromeTest {

    WebDriver driver;

    WebDriverManager wdm = WebDriverManager.chromedriver().browserInDocker();
}

```

```

@BeforeEach
void setupTest() {
    driver = wdm.create();
}

@AfterEach
void teardown() {
    wdm.quit();
}

@Test
void test() {
    driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
    assertThat(driver.getTitle()).contains("Selenium WebDriver");
}
}

```



When using browsers in Docker containers, the call to the method `quit()` of the WebDriverManager API allows the disposal of the browser container(s).

As of version 6, WebDriverManager uses the official Docker browser images created and maintained by the Selenium project, i.e., [docker-selenium](#). Therefore, the available browsers to be executed as Docker containers in WebDriverManager are Chrome (like the previous example), [Firefox](#), [Edge](#), and [Chromium](#).



Some *dockerized* browsers (e.g., Chromium and Edge) are supported in ARM64 architecture. See the [docker-selenium](#) README for further information.

3.4.1. Browser Versions

A significant aspect of the browser containers presented so far is that WebDriverManager connects to Docker Hub to discover the latest available release when the browser version is not specified (like the examples explained before). This way, the *dockerized* browsers of tests handled by WebDriverManager are auto-maintained, in the sense that these tests use the latest version available without any additional effort.

Nevertheless, we can force a given browser version in Docker using the method `browserVersion()` of the WebDriverManager API. This method accepts a `String` parameter specifying the version. This version can be fixed (e.g., `132.0`), and it also accepts the following wildcards:

- `"latest"`: To specify the latest version explicitly (default option).
- `"beta"`: To use the beta version.
- `"dev"`: To use the development version.

The following example shows a test using Chrome beta in Docker (see a similar example using Firefox dev [here](#)).

```

import static io.github.bonigarcia.wdm.WebDriverManager.isDockerAvailable;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assumptions.assumeThat;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class DockerChromeBetaTest {

    WebDriver driver;

    WebDriverManager wdm = WebDriverManager.chromedriver().browserInDocker()
        .browserVersion("beta");

    @BeforeEach
    void setupTest() {
        assumeThat(isDockerAvailable()).isTrue();
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        wdm.quit();
    }

    @Test
    void test() {
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        assertThat(driver.getTitle()).contains("Selenium WebDriver");
    }

}

```

3.4.2. Remote Desktop

A possible inconvenience of using browsers in Docker is that we cannot see what is happening inside the container by default. To improve this situation, WebDriverManager allows connecting to the remote desktop session simply by invoking the method `enableVnc()` of a dockerized browser. When using this option, two different technologies are used internally:

- Virtual Network Computing (**VNC**), a graphical desktop sharing system. In WebDriverManager, a VNC server is started in the browser container.
- **noVNC**, a open-source web-based VNC client. In WebDriverManager, a custom **noVNC Docker image** is used to connect through noVNC.

The following example shows a test that enables this feature. WebDriverManager writes the noVNC URL in the **INFO** trace logs. In addition, as shown in this example, this URL can be found by invoking the method `getDockerNoVncUrl()`. We can use this URL to inspect and interact with the browser during the test execution (as shown in the following picture).

```
import static org.assertj.core.api.Assertions.assertThat;

import java.net.URL;
import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class DockerChromeVncTest {

    WebDriver driver;

    WebDriverManager wdm = WebDriverManager.chromedriver().browserInDocker()
        .enableVnc();

    @BeforeEach
    void setupTest() {
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        wdm.quit();
    }

    @Test
    void test() throws Exception {
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        assertThat(driver.getTitle()).contains("Selenium WebDriver");

        // Verify URL for remote session
        URL noVncUrl = wdm.getDockerNoVncUrl();
        assertThat(noVncUrl).isNotNull();

        // Pause for manual inspection
        Thread.sleep(Duration.ofSeconds(60).toMillis());
    }
}
```

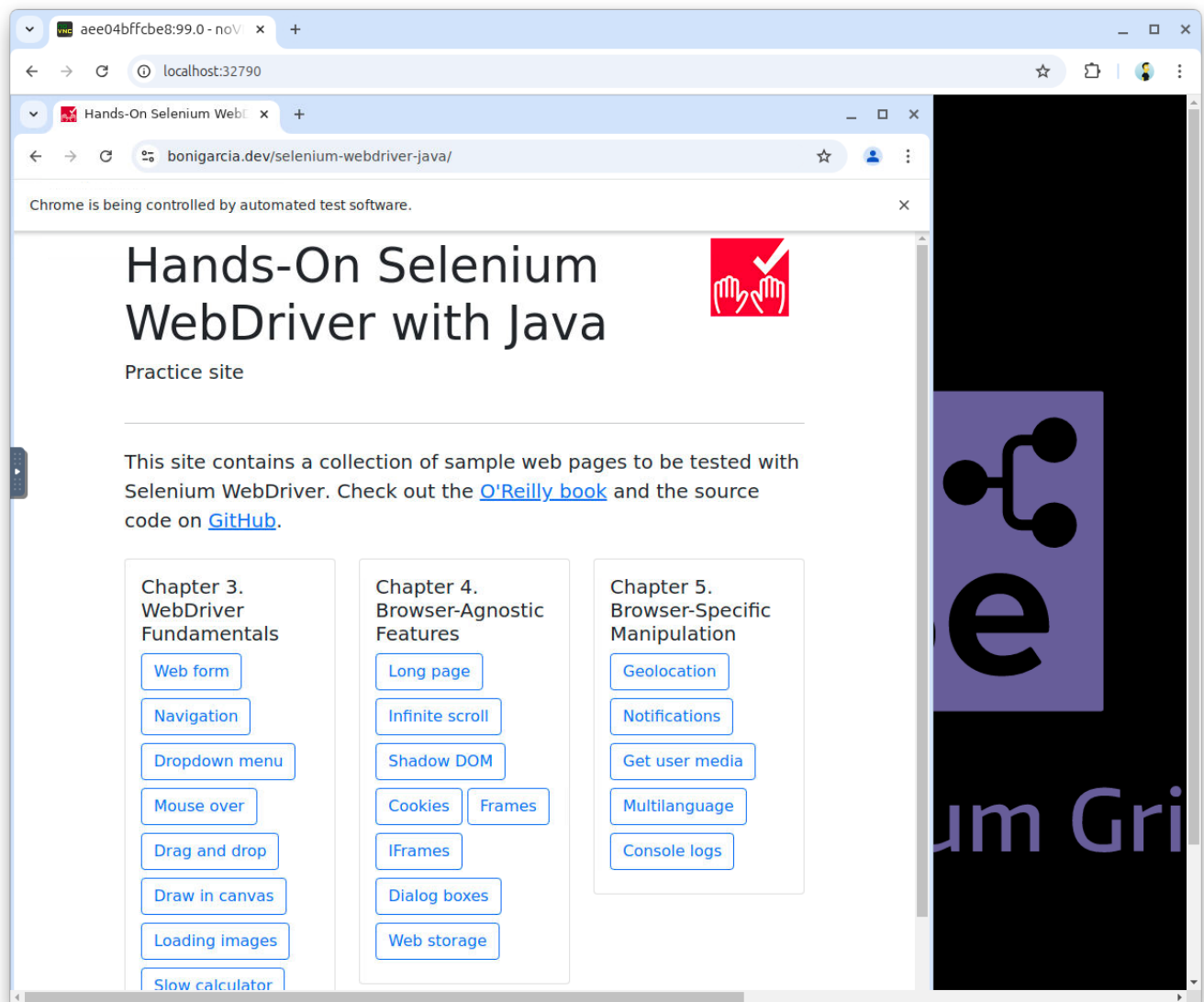


Figure 2. Example of noVNC session using Chrome in Docker

3.4.3. Video Recording

The following related feature is recording the desktop session of a dockerized browser. To enable it, we need to invoke the method `enableRecording()` in `WebDriverManager`. Internally, `WebDriverManager` starts another Docker container using `FFmpeg` to record the browser session. At the end of the test, we can find the recording in MP4 (by default, with a filename composed of the browser name followed by the symbol `_` and the system timestamp, plus another `_` and the session id) located in the project root folder (you can change this behavior using [configuration capabilities](#)). The following test shows an example of this feature.

```
import static org.assertj.core.api.Assertions.assertThat;

import java.nio.file.Path;
import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
```

```

import org.openqa.selenium.WebDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

class DockerChromeRecordingTest {

    WebDriver driver;

    WebDriverManager wdm = WebDriverManager.chromedriver().browserInDocker()
        .enableRecording();

    @BeforeEach
    void setupTest() {
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        wdm.quit();
    }

    @Test
    void test() throws Exception {
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        assertThat(driver.getTitle()).contains("Selenium WebDriver");

        // Pause to see the navigation in the recording
        Thread.sleep(Duration.ofSeconds(2).toMillis());

        driver.findElement(By.partialLinkText("form")).click();

        // Pause to generate a longer recording
        Thread.sleep(Duration.ofSeconds(2).toMillis());

        // Verify recoding file
        Path recordingPath = wdm.getDockerRecordingPath();
        assertThat(recordingPath).exists();

        // Get recording as base64
        String recordingBase64 = wdm.getRecordingBase64();
        assertThat(recordingBase64).isNotEmpty();
    }
}

```

3.4.4. Advanced Settings

The dockerized browser provided by WebDriverManager can be configured in different ways. For example, the WebDriverManager API allows using volumes, customizing the language and timezone inside the browser container, using custom images, configuring remote Docker daemon,

customizing shared memory and in-memory filesystem (*tmpfs*), changing the screen resolution and video frame rate, or customizing the recording output (folder and filename). See the [advanced configuration](#) section for specific details about it.

In addition, the WebDriverManager API provides several methods to get the most of the dockerized browsers, namely:

- `getDockerRecordingPath()`: Get path of the session recording.
- `getDockerNoVncUrl()`: Ger URL of the remote desktop noVNC session.
- `getDockerSeleniumServerUrl()`: Get the URL of the underlying Selenium Server (inside the container) that allows controlling the remote (dockerized) browser.
- `getDockerService()`: It allows access to the Docker service and client (based on [docker-java](#)) to make custom operations with Docker containers (e.g., run commands in the browser container, see example [here](#)).
- `getDockerBrowserContainerId()`: Get browser container id (required for advance operation using the Docker client)
- `getWebDriver()`: Get the previously created `WebDriver` object (the same as the returned by the method `create()`).
- `getWebDriverList`: Get the previously created `WebDriver` objects (if any).

A manager instance can be used to create more than one `WebDriver` object. For this reason, the methods `getDockerRecordingPath()`, `getDockerNoVncUrl()`, `getDockerSeleniumServerUrl()`, `getDockerBrowserContainerId()`, and `quit()` are overloaded, allowing to specify an `WebDriver` instance. When no parameter is specified in these methods, WebDriverManager returns the first `WebDriver` object (this is a usual case, i.e., a manager creates a single instance of `WebDriver`). You can see an example of a manager used to create more than one browser [here](#).

3.5. Browsers Monitoring

As of version 5.2.0, WebDriverManager provides seamless integration with [BrowserWatcher](#). BrowserWatcher is a browser extension designed to monitor different aspects of web browsers such as Chrome, Firefox, or Edge. This section summarizes the features of BrowserWatcher integrated into WebDriverManager.

3.5.1. Console Log Gathering

Gathering the browser console might help find the cause of failed Selenium tests. BrowserWatcher provides a cross-browser mechanism based on JavaScript to implement this feature. WebDriverManager incorporates this feature for browsers controlled with Selenium WebDriver created through the WebDriverManager method `create()` and decorated with `watch()`. Internally, BrowserWatcher is installed as a browser extension and starts gathering browser logs during the session time. At some point, we need to invoke the WebDriverManager method `getLogs()` to get the collected logs from the Java logic. The following test shows a basic example of this feature.

```
import static java.lang.invoke.MethodHandles.lookup;
import static org.assertj.core.api.Assertions.assertThat;
```

```

import static org.slf4j.LoggerFactory.getLogger;

import java.util.List;
import java.util.Map;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.slf4j.Logger;

import io.github.bonigarcia.wdm.WebDriverManager;

class GatherLogsFirefoxTest {

    static final Logger log = getLogger(lookup().lookupClass());

    WebDriverManager wdm = WebDriverManager.firefoxdriver().watch();
    WebDriver driver;

    @BeforeEach
    void setup() {
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        driver.quit();
    }

    @Test
    void test() {
        driver.get(
            "https://bonigarcia.dev/selenium-webdriver-java/console-logs.html");

        List<Map<String, Object>> logMessages = wdm.getLogs();

        assertThat(logMessages).hasSize(5);

        logMessages.forEach(map -> log.debug("{} {} {}",
            map.get("datetime"),
            String.format("%1$-14s",
                map.get("source").toString().toUpperCase() + "."
                + map.get("type").toString().toUpperCase()),
            map.get("message")));
    }
}

```



When using Firefox, this feature requires at least Selenium WebDriver 4.1.2.

3.5.2. Console Log Displaying

In addition to log gathering, BrowserWatcher allows displaying the console logs as dialog notifications on the page. This feature can be enabled using the method `watchAndDisplay()`, for example, as follows:

```
import static java.lang.invoke.MethodHandles.lookup;
import static org.assertj.core.api.Assertions.assertThat;
import static org.slf4j.LoggerFactory.getLogger;

import java.time.Duration;
import java.util.List;
import java.util.Map;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.slf4j.Logger;

import io.github.bonigarcia.wdm.WebDriverManager;

class DisplayLogsChromeTest {

    static final Logger log = getLogger(lookup().lookupClass());

    WebDriverManager wdm = WebDriverManager.chromedriver().watchAndDisplay();
    WebDriver driver;

    @BeforeEach
    void setup() {
        driver = wdm.create();
    }

    @AfterEach
    void teardown() throws InterruptedException {
        // pause for manual browser inspection
        Thread.sleep(Duration.ofSeconds(3).toMillis());

        driver.quit();
    }

    @Test
    void test() {
        driver.get(
            "https://bonigarcia.dev/selenium-webdriver-java/console-logs.html");

        List<Map<String, Object>> logMessages = wdm.getLogMessages();

        assertThat(logMessages).hasSize(5);
    }
}
```

```

logMessages.forEach(map -> log.debug("{} {} {}",
    map.get("datetime"),
    String.format("%1$-14s",
        map.get("source").toString().toUpperCase() + "."
        + map.get("type").toString().toUpperCase()),
    map.get("message")));
}

}

```

The following picture shows an example of these notifications:

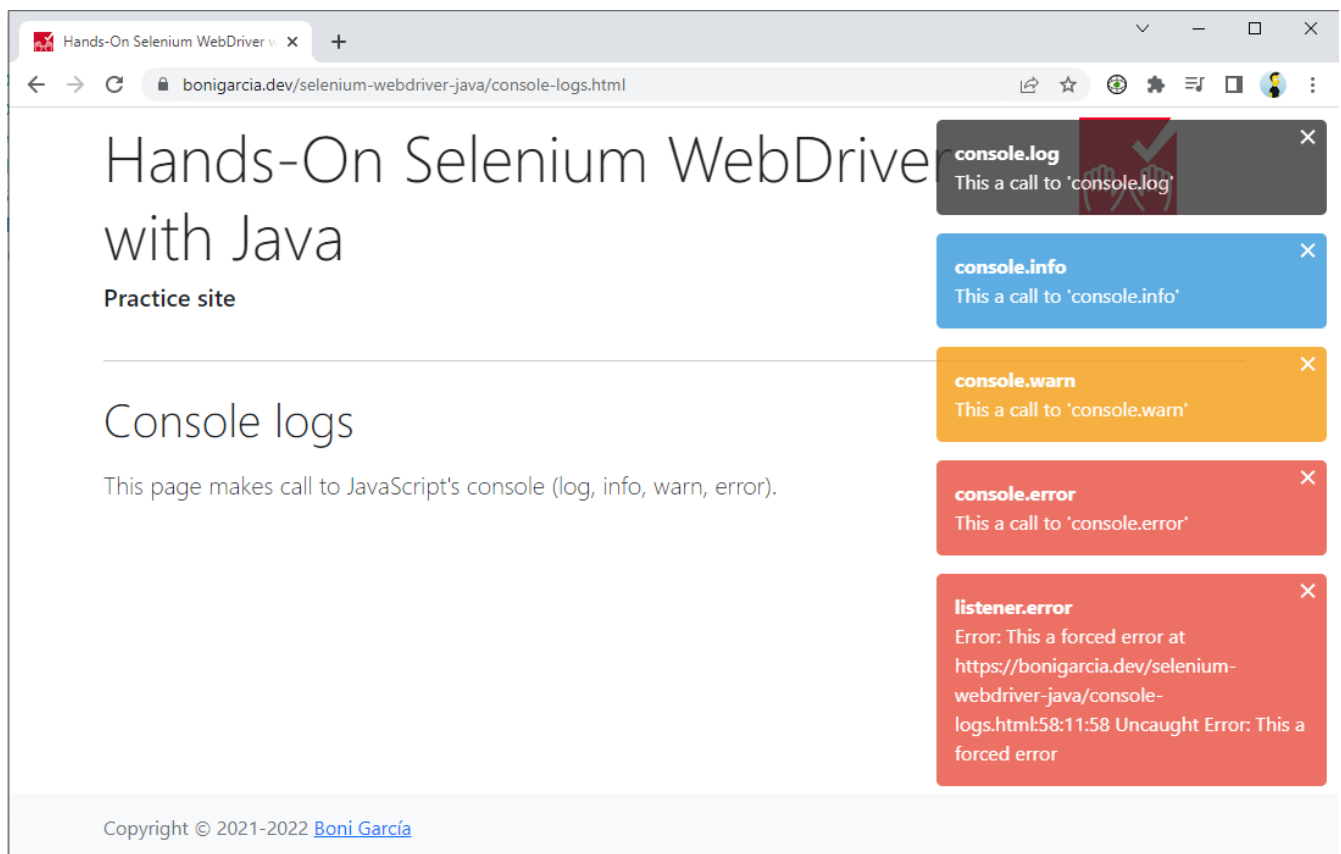


Figure 3. Display logs example



This feature is not available for BrowserWatcher 2.x (i.e., based on manifest v3 -MV3-). Therefore, this feature is currently only available for Firefox.

3.5.3. Video Recording

BrowserWatcher (MV3) allows recording the current browser tab (i.e., the browser viewport). This feature requires a page loaded in the current tab (in other words, it cannot record empty or configuration pages). Then, the recording is started using the method `startRecording()` and stopped with `stopRecording()`. After stopping, the video recording will be automatically downloaded by the browser, and the recording (a webm file) will be stored in the downloads folder. You can discover the path of this recording using the method `getRecordingPath()`. Alternatively, you can get the recording in Base 64 using the method `getRecordingBase64()`. The following test illustrates this

feature:

```
import static java.lang.invoke.MethodHandles.lookup;
import static org.assertj.core.api.Assertions.assertThat;
import static org.slf4j.LoggerFactory.getLogger;

import java.nio.file.Path;
import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.slf4j.Logger;

import io.github.bonigarcia.wdm.WebDriverManager;

@Disabled("Broken in Chrome 142")
class RecordChromeTest {

    static final Logger log = getLogger(lookup().lookupClass());

    WebDriver driver;
    WebDriverManager wdm;

    @BeforeEach
    void setup() {
        wdm = WebDriverManager.chromedriver().watch();
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        driver.quit();
    }

    @Test
    void test() {
        driver.get(
            "https://bonigarcia.dev/selenium-webdriver-java/slow-calculator.html"
        );

        wdm.startRecording();

        // 1 + 3
        driver.findElement(By.xpath("//span[text()='1']")).click();
```

```

driver.findElement(By.xpath("//span[text()='+' ]")).click();
driver.findElement(By.xpath("//span[text()='3' ]")).click();
driver.findElement(By.xpath("//span[text()='=' ]")).click();

// ... should be 4, wait for it
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.textToBe(By.className("screen"), "4"));

wdm.stopRecording();

// Get recording as path
Path recordingPath = wdm.getRecordingPath();
assertThat(recordingPath).exists();
log.debug("Recording available at {}", recordingPath);

// Get recording as base64
String recordingBase64 = wdm.getRecordingBase64();
assertThat(recordingBase64).isNotEmpty();
}
}

```



This feature is based on the [tabCapture API](#). Therefore, it is only available in Chromium-based browsers (e.g., Chrome and Edge).

3.5.4. Disabling CSP

[Content Security Policy \(CSP\)](#) is the name of an HTTP response header that browsers use to improve the security of web pages. CSP helps to protect from attacks such as cross-site scripting (XSS). Nevertheless, developers might want to disable the CSP headers received from the server for testing purposes. For this reason, BrowserWatcher allows bypassing these CSP headers. This feature (i.e., disabling CSP headers) can be enabled in WebDriverManager using the method `disableCsp()`, as follows:

```

import static java.lang.invoke.MethodHandles.lookup;
import static org.assertj.core.api.Assertions.assertThat;
import static org.slf4j.LoggerFactory.getLogger;

import java.util.List;
import java.util.Map;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.slf4j.Logger;

import io.github.bonigarcia.wdm.WebDriverManager;

```

```

class DisableCspFirefoxTest {

    static final Logger log = getLogger(lookup().lookupClass());

    WebDriverManager wdm = WebDriverManager.firefoxdriver().watch()
        .disableCsp();
    WebDriver driver;

    @BeforeEach
    void setup() {
        driver = wdm.create();
    }

    @AfterEach
    void teardown() {
        driver.quit();
    }

    @Test
    void test() {
        driver.get("https://paypal.com/");
        List<Map<String, Object>> logMessages = wdm.getLogs();
        assertThat(logMessages).isNotNull();
    }
}

```



This feature is currently only available for Firefox.

Chapter 4. Other Usages

In addition to as a regular Java dependency, WebDriverManager can be used in other use cases. This section summarizes these usages.

4.1. WebDriverManager CLI

WebDriverManager can be used interactively from the Command Line Interface (CLI), i.e., the shell. There are three different ways to use WebDriverManager as a CLI tool:

1. Using the WebDriverManager *fat-JAR* (i.e., WebDriverManager with all its dependencies in a single executable JAR file). This JAR file is generated from the source using the Maven command `mvn compile assembly:single`, and it is released on GitHub with every new version of WebDriverManager. You can download the latest of this fat-JAR from [here](#). Once you get this file, you need to use the following command in the shell (where `<args>` are the accepted arguments, explained below):

```
java -jar webdrivermanager-6.4.0-fat.jar <args>
```

2. Using the source code. WebDriverManager is hosted on [GitHub](#). We can use Maven to manage its Java source code. For example, to run the CLI mode using Maven and the source code, we need to invoke the following Maven command in the shell from the project root:

```
mvn exec:java -Dexec.args="<args>"
```

3. Using the WebDriverManager Docker container. Each new release of WebDriverManager is pushed to [Docker Hub](#) as a container based on [OpenJDK](#) plus the WebDriverManager fat-JAR. The default command to run the WebDriverManager Docker container is described below.

```
docker run --rm -e ARGS="<args>" bonigarcia/webdrivermanager:6.4.0
```

WebDriverManager CLI can be used for three different purposes. We can see these options by launching the CLI with empty or invalid arguments (`<args>` in the commands before). In this case, the output of WebDriverManager CLI is the following:

```
[ERROR] The valid arguments for WebDriverManager CLI are:
[ERROR] 1. For resolving drivers locally:
[ERROR]     resolveDriverFor browserName <browserVersion>
[ERROR] (where browserName is: chrome|edge|firefox|opera|chromium|iexplorer)
[ERROR] 2. For running a browser in a Docker (and use it trough noVNC):
[ERROR]     runInDocker browserName <browserVersion>
[ERROR] (where browserName is: chrome|edge|firefox|chromium)
[ERROR] 3. For starting WebDriverManager Server:
```

```
[ERROR]          server <port>
[ERROR] (where the default port is 4444)
```

Option 1: Driver Resolver

WebDriverManager CLI can be used to resolve drivers (e.g., chromedriver, geckodriver) by applying the usual [resolution algorithm](#) from the shell. This feature can be interesting if we want to download drivers outside a Java program. To use this option, we need to invoke WebDriverManager CLI using the following arguments (supposing we need to resolve chromedriver):

- Fat-JAR:

```
java -jar webdrivermanager-6.4.0-fat.jar resolveDriverFor chrome
```

- Source code:

```
mvn exec:java -Dexec.args="resolveDriverFor chrome"
```

- Docker container:

```
docker run --rm -v ${PWD}:/wdm -e ARGS="resolveDriverFor chrome"
bonigarcia/webdrivermanager:6.4.0
```

Option 2: Browsers in Docker

WebDriverManager CLI can execute browsers in Docker containers and interact with them using noVNC. This feature can be interesting for *exploratory testing* for web applications using different types and versions of web browsers. To use this option, the arguments we need to use are the following (supposing we want to use Chrome):

- Fat-JAR:

```
java -jar webdrivermanager-6.4.0-fat.jar runInDocker chrome
```

- Source code:

```
mvn exec:java -Dexec.args="runInDocker chrome"
```

- Docker container:

```
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock -e ARGS="runInDocker
```

```
chrome" bonigarcia/webdrivermanager:6.4.0
```

Option 3: Server

Finally, WebDriverManager CLI allows starting the WebDriverManager Server, as follows (see [next section](#) for more details about it):

- Fat-JAR:

```
java -jar webdrivermanager-6.4.0-fat.jar server
```

- Source code:

```
mvn exec:java -Dexec.args="server"
```

- Docker container:

```
docker run --rm -p 4444:4444 -v /var/run/docker.sock:/var/run/docker.sock  
bonigarcia/webdrivermanager:6.4.0
```

4.2. WebDriverManager Server

The WebDriverManager Server is based on HTTP and offers two types of services. First, it can resolve drivers (chromedriver, geckodriver, etc.). The WebDriverManager Server exposes a simple REST-like API to this aim. WebDriverManager Server sends the resolved driver as an HTTP attachment in the response. The endpoints provided by this API are the following (supposing that WebDriverManager is running the localhost in its default port, i.e., 4444):

- <http://localhost:4444/chromedriver>: To resolve chromedriver.
- <http://localhost:4444/firefoxdriver>: To resolve geckodriver.
- <http://localhost:4444/edgedriver>: To resolve msedgedriver.
- <http://localhost:4444/operaedriver>: To resolve geckodriver.
- <http://localhost:4444/iedriver>: To resolve geckodriver.



You can parametrize these URLs using all the configuration keys available in WebDriverManager (see the [advanced configuration](#) section) as parameters, but removing the **wdm.** prefix. For instance, the URL for requesting the resolution of chromedriver for Chrome 100 would be <http://localhost:4444/chromedriver?chromeVersion=100>.

Second, the WebDriverManager Server acts as a regular Selenium Server (i.e., a *hub* in the classical Selenium Grid architecture). This feature can create remote **WebDriver** instances using the WebDriverManager Server (even for different language bindings than Java). The following example

shows a Node.js test using Selenium WebDriver and WebDriverManager Server (notice that by default, the WebDriverManager Server URL does not require any path, i.e., <http://localhost:4444/>):

```
var webdriver = require("selenium-webdriver");

async function wdmServerTest() {
  var wdmServerUrl = "http://localhost:4444/";
  var capabilities = {
    browserName : "chrome",
    version: "100"
  };

  try {
    var driver = await new webdriver.Builder().usingServer(wdmServerUrl)
      .withCapabilities(capabilities).build();

    var sutUrl = "https://bonigarcia.dev/selenium-webdriver-java/";
    await driver.get(sutUrl);

    await driver.getTitle().then(function(title) {
      console.log("The title of " + sutUrl + " is '" + title + "'");
    });

  } catch (err) {
    console.error("Something went wrong!\n", err.stack);
  } finally {
    if (driver) {
      driver.quit();
    }
  }
}

wdmServerTest();
```

4.3. WebDriverManager Agent

WebDriverManager can also be used as *Java Agent*. In this case, and using the JVM instrumentation API, WebDriverManager intercepts calls to applications running on the JVM and modifies their bytecode. In particular, the WebDriverManager Agent uses this technique to check the objects being created in the JVM. Before Selenium WebDriver objects are instantiated (*ChromeDriver*, *FirefoxDriver*, etc.), the required manager is used to resolve its driver (chromedriver, geckodriver, etc.). Thanks to this approach, we can get rid of the WebDriverManager call (e.g. *WebDriverManager.chromedriver.setup()*;) from our test, for example:

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

class ChromeAgentTest {

    WebDriver driver;

    @BeforeEach
    void setupTest() {
        driver = new ChromeDriver();
    }

    @AfterEach
    void teardown() {
        if (driver != null) {
            driver.quit();
        }
    }

    @Test
    void test() {
        // Test logic
    }

}

```

To configure the WebDriverManager Agent, we need to specify the path of the WebDriverManager **fat-JAR** using the JVM flag `-javaagent:/path/to/webdrivermanager-6.4.0-fat.jar`. Alternatively, it can be done using Maven (see a complete project example [here](#)).

4.4. Selenium-Jupiter

WebDriverManager is the heart of the project [Selenium-Jupiter](#), an open-source JUnit 5 extension for Selenium WebDriver. Selenium-Jupiter uses the programming and extension model provided by JUnit 5 (named **Jupiter**) together with WebDriverManager to create tests with reduced boilerplate code. For instance, thanks to the Jupiter feature for parameter resolution, we can declare a type of the **WebDriver** hierarchy (e.g., **ChromeDriver**) as a test parameter. Internally, Selenium-Jupiter resolves its driver and creates the instance before tests, and then the browser is gracefully closed at the end of the test. For example:

```

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.openqa.selenium.chrome.ChromeDriver;

import io.github.bonigarcia.seljup.SeleniumJupiter;

@ExtendWith(SeleniumJupiter.class)

```

```
class ChromeTest {

    @Test
    void test(ChromeDriver driver) {
        driver.get("https://bonigarcia.dev/selenium-webdriver-java/");
        assertThat(driver.getTitle()).contains("Selenium WebDriver");
    }

}
```

Selenium-Jupiter provides more different features, such as seamless integration with Docker, test templates (for cross-browser testing), conditional test execution (depending on the availability of browsers), conditional screenshots and recordings (when tests fail), and more. For further details, read the [Selenium-Jupiter documentation](#).

4.5. Selenium Grid

[Selenium Grid](#) is an infrastructure that allows serving remote browsers to be used in Selenium WebDriver tests. The nodes in which the browsers are executed in Selenium Grid also require managing the required drivers (chromedriver, geckodriver, etc.). To manage these drivers, as usual, we can use WebDriverManager.

For example, this [test](#) shows how to start a Selenium Grid in standalone mode, resolving the driver with WebDriverManager and registering a Chrome browser in the Selenium Server. In addition, if we start Selenium Grid from the shell, we can complement it with WebDriverManager CLI to resolve the required drivers, as follows:

```
boni@ubuntu:~$ java -jar webdrivermanager-6.4.0-fat.jar resolveDriverFor chrome
[INFO] Using WebDriverManager to resolve chrome
[DEBUG] Detecting chrome version using online commands.properties
[DEBUG] Running command on the shell: [google-chrome, --version]
[DEBUG] Result: Google Chrome 103.0.5060.134
[DEBUG] Latest version of chromedriver according to
https://chromedriver.storage.googleapis.com/LATEST_RELEASE_103 is 103.0.5060.134
[INFO] Using chromedriver 103.0.5060.134 (resolved driver for Chrome 103)
[INFO] Reading https://chromedriver.storage.googleapis.com/ to seek chromedriver
[DEBUG] Driver to be downloaded chromedriver 103.0.5060.134
[INFO] Downloading
https://chromedriver.storage.googleapis.com/103.0.5060.134/chromedriver_linux64.zip
[INFO] Extracting driver from compressed file chromedriver_linux64.zip
[INFO] Driver location: /home/boni/chromedriver

boni@ubuntu:~$ java -jar selenium-server-4.3.0.jar standalone
01:31:52.806 INFO [LoggingOptions.configureLogEncoding] - Using the system default
encoding
01:31:52.810 INFO [OpenTelemetryTracer.createTracer] - Using OpenTelemetry for tracing
01:31:53.344 INFO [NodeOptions.getSessionFactories] - Detected 16 available processors
01:31:53.359 INFO [NodeOptions.discoverDrivers] - Discovered 1 driver(s)
01:31:53.382 INFO [NodeOptions.report] - Adding Chrome for {"browserName": "chrome"}
```

```
16 times
01:31:53.417 INFO [Node.<init>] - Binding additional locator mechanisms: id, relative,
name
01:31:53.436 INFO [GridModel.setAvailability] - Switching Node 3804a261-5889-44c5-
8cb1-d56162cf39ef (uri: http://172.18.0.1:4444) from DOWN to UP
01:31:53.436 INFO [LocalDistributor.add] - Added node 3804a261-5889-44c5-8cb1-
d56162cf39ef at http://172.18.0.1:4444. Health check every 120s
01:31:53.554 INFO [Standalone.execute] - Started Selenium Standalone 4.3.0 (revision
a4995e2c09*): http://172.18.0.1:4444
```

4.6. Appium

[Appium](#) is a test automation framework for mobile applications. In the same way that Selenium WebDriver, Appium also speaks the W3C WebDriver protocol to drive web browsers on mobile devices. For this reason, the required driver must be present to control mobile browsers with Appium.

Again, WebDriverManager can help in this task. The following snippet shows a Java method that creates a `WebDriver` object using the driver path for the Chrome browser on an Android device:

```
public WebDriver createChromeAndroidDriver(String browserVersion,
    String deviceName, URL appiumServerUrl) {
    // Resolve driver and get its path
    WebDriverManager wdm = WebDriverManager.chromedriver()
        .browserVersion(browserVersion);
    wdm.setup();
    String chromedriverPath = wdm.getDownloadedDriverPath();

    // Create WebDriver instance using the driver path
    DesiredCapabilities capabilities = new DesiredCapabilities();
    capabilities.setCapability("browserName", "chrome");
    capabilities.setCapability("version", browserVersion);
    capabilities.setCapability("deviceName", deviceName);
    capabilities.setCapability("platformName", "android");
    capabilities.setCapability("chromedriverExecutable", chromedriverPath);

    return new AndroidDriver<WebElement>(appiumServerUrl, capabilities);
}
```

Chapter 5. Examples

All the examples presented in this documentation are available in the [WebDriverManager tests](#). Moreover, different public repositories contain test examples using WebDriverManager, such as:

- [WebDriverManager Basic](#): A simple project using JUnit 5, Selenium WebDriver, and WebDriverManager.
- [WebDriverManager Examples](#): Different examples with JUnit 5, Selenium WebDriver, and WebDriverManager.
- [WebDriverManager Spring-Boot](#): Another simple project using Spring-Boot, JUnit 5, Selenium WebDriver, and WebDriverManager.
- [WebDriverManager Agent Example](#): Maven project using WebDriverManager as Agent.
- [Selenium WebDriver with Java](#): A comprehensive collection of Selenium WebDriver 4 examples using Java as language binding.

Chapter 6. Advanced Configuration

WebDriverManager provides different ways of configuration. First, by using its *Java API*. To that aim, each manager (e.g., `chromedriver()`, `firefoxdriver()`, etc., allows for concatenating different methods of this API to specify custom options or preferences. For example (the explanation of these methods and the other possibilities are explained in the tables at the end of this section):

```
WebDriverManager.chromedriver().driverVersion("81.0.4044.138").setup();
WebDriverManager.firefoxdriver().browserVersion("75").setup();
WebDriverManager.operadriver().proxy("server:port").setup();
WebDriverManager.edgedriver().mac().setup();
```



In addition to the methods presented in this section, each manager provides a `config()` method that configures all the possible parameters of WebDriverManager. In addition, WebDriverManager provides the method `reset()` to restore to the default values the parameters of a given manager.

The second alternative to tune WebDriverManager is using *Java system properties*. In this method, each WebDriverManager API method has its equivalence of a unique *configuration key*. For instance, the API method `cachePath()` (used to specify the driver cache folder) is equivalent to the configuration key `wdm.cachePath`. These types of configuration keys can be passed when executing the tests, for example, using Maven:

```
mvn test -Dwdm.cachePath=/custom/path/to/driver/cache
```

The third way to configure WebDriverManager is using *environmental variables*. The names for these variables are made by converting each configuration key name (e.g., `wdm.cachePath`) to uppercase and replacing the symbol `.` with `_` (e.g., `WDM_CACHEPATH`). These variables can be helpful to global setup parameters at the operating system level. Also, it allows specifying a custom setup when using WebDriverManager as a Docker container. For example:

```
docker run --rm -v ${PWD}:/wdm -e ARGS="resolveDriverFor chrome" -e
WDM_CHROMEVERSION=84 bonigarcia/webdrivermanager:6.4.0
```



The preference order of these configuration alternatives is (in case of overlapping) is: 1) Environmental Variables. 2) Java system properties. 3) Java API.

The remainder of this section describes all the possible Java methods in the WebDriverManager API and its equivalent configuration keys in three groups of capabilities: [driver management](#), [browsers in Docker](#), and [WebDriverManager Server](#).

Table 1. Configuration capabilities for driver management

API method	Configuration key	Default value	Description
cachePath(String)	wdm.cachePath	~/.cache/selenium	Folder to store drivers locally
resolutionCachePath(String)	wdm.resolutionCachePath	~/.cache/selenium	Folder to store the resolution cache
driverVersion(String)	wdm.chromeDriverVersion, wdm.operaDriverVersion, wdm.iExplorerDriverVersion, wdm.edgeDriverVersion, wdm.geckoDriverVersion, wdm.chromiumDriverVersion	"" (automatic driver version discovery through the resolution algorithm)	Custom driver version
browserVersion(String)	wdm.chromeVersion, wdm.operaVersion, wdm.edgeVersion, wdm.firefoxVersion, wdm.chromiumVersion, wdm.safariVersion	"" (automatic browser version detection using the commands database)	Custom browser version (major)
browserBinary(String)	wdm.chromeBinary, wdm.operaBinary, wdm.edgeBinary, wdm.firefoxBinary, wdm.chromiumBinary	"" (automatic browser version detection using the default browser locations as specified in the commands database)	Custom browser binary path
forceDownload()	wdm.forceDownload=true	false (drivers in cache are reused if available)	Force downloading driver (even if it is already in the cache)
useBetaVersions()	wdm.useBetaVersions=true	false (driver versions are skipped)	Allow the use beta versions (if possible)
architecture(Architecture)	wdm.architecture	"" (automatic architecture discovery)	Force a given architecture for a driver
arch32()	wdm.architecture=X32	"" (automatic architecture discovery)	Use 32-bit driver version
arch64()	wdm.architecture=X64	"" (automatic architecture discovery)	Use 64-bit driver version
arm64()	wdm.architecture=ARM64	"" (automatic architecture discovery)	Use ARM (Aarch) 64-bit driver version
operatingSystem(Operating System)	wdm.os	"" (automatic OS discovery)	Force a given operating system (WIN , LINUX , or MAC)

API method	Configuration key	Default value	Description
<code>win()</code>	<code>wdm.os=WIN</code>	<code>""</code> (automatic OS discovery)	Force Windows
<code>linux()</code>	<code>wdm.os=LINUX</code>	<code>""</code> (automatic OS discovery)	Force Linux
<code>mac()</code>	<code>wdm.os=MAC</code>	<code>""</code> (automatic OS discovery)	Force Mac OS
<code>driverRepositoryUrl(URL)</code>	<code>wdm.chromeDriverUrl</code> , <code>wdm.operaDriverUrl</code> , <code>wdm.edgeDriverUrl</code> , <code>wdm.geckoDriverUrl</code> , <code>wdm.iExplorerDriverUrl</code>	Driver specific URLs (available in webdrivermanager.properties)	Change the repository URL in which the drivers are hosted
<code>useMirror()</code>	<code>wdm.useMirror=true</code>	<code>false</code> (not using mirrors)	Enable the use of a driver repository mirror (available for chromedriver, geckodriver, and operadriver)
<code>proxy(String)</code>	<code>wdm.proxy</code>	<code>""</code> (no proxy)	Use an HTTP proxy for the network connection (using the notation <code>proxy:port</code> or <code>username:password@proxy:port</code>). It can be also configured using the environment variable <code>HTTPS_PROXY</code>
<code>proxyUser(String)</code>	<code>wdm.proxyUser</code>	<code>""</code> (no proxy user)	Username for the HTTP proxy. It can be also configured using the environment variable <code>HTTPS_PROXY_USER</code>
<code>proxyPass(String)</code>	<code>wdm.proxyPass</code>	<code>""</code> (no proxy password)	Password for HTTP proxy. It can be also configured using the environment variable <code>HTTPS_PROXY_PASS</code>
<code>gitHubToken(String)</code>	<code>wdm.gitHubToken</code>	<code>""</code> (no GitHub token)	Personal access token for authenticated GitHub requests (see known issues). It can be also configured using the environment variable <code>GITHUB_TOKEN</code>

API method	Configuration key	Default value	Description
<code>ignoreVersions(String ...)</code>	<code>wdm.ignoreVersions</code>	<code>""</code> (no ignored versions)	Ignore specific driver version(s)
<code>timeout(int)</code>	<code>wdm.timeout</code>	<code>30</code>	Timeout (in seconds) to connect and download drivers from online repositories
<code>properties(String)</code>	<code>wdm.properties</code>	<code>webdrivermanager.properties</code>	Properties file (in the project classpath) for default configuration values
<code>avoidExport()</code>	<code>wdm.avoidExport=true</code>	<code>false</code> (export driver paths as Java properties (e.g. <code>webdriver.chrome.driver</code>))	Avoid step 4 in the resolution algorithm (for instance, in the CLI mode)
<code>exportParameter(String)</code> <code>exportParameter(DriverManagerType)</code>	<code>wdm.chromeDriverExport</code> , <code>wdm.geckoDriverExport</code> , <code>wdm.edgeDriverExport</code> , <code>wdm.iExplorerDriverExport</code> , <code>wdm.operaDriverExport</code>	Java property name used to export the driver path (available in webdrivermanager.properties)	Set custom property name. An <code>DriverManagerType</code> enum (e.g., <code>CHROME</code> , <code>FIREFOX</code> , etc.) can be used instead of the <code>String</code> parameter
<code>avoidOutputTree()</code>	<code>wdm.avoidOutputTree=true</code>	<code>false</code> (create output tree in the driver cache, e.g., <code>chromedriver/linux64/2.37</code>)	Avoid output tree (for instance, in the CLI mode)
<code>avoidFallback()</code>	<code>wdm.avoidFallback=true</code>	<code>false</code> (use a retries mechanism if any problem happens during the resolution algorithm)	Avoid the fallback mechanism
<code>avoidBrowserDetection()</code>	<code>wdm.avoidBrowserDetection=true</code>	<code>false</code> (browser version is detected, and the corresponding driver version is discovered)	Force to use the latest version available for a given driver

API method	Configuration key	Default value	Description
<code>avoidTmpFolder()</code>	<code>wdm.avoidTmpFolder=true</code>	<code>false</code> (Each driver release (typically compressed) is copied in a temporal folder in the local machine, and then the driver is extracted and copied to the driver cache)	Avoid using a temporal folder to download drivers (and handle driver release directly on driver cache)
<code>avoidShutdownHook()</code>	<code>wdm.avoidShutdownHook=true</code>	<code>false</code>	Avoid shutdown hook for drivers objected created with <code>create()</code>
<code>avoidExternalConnections()</code>	<code>wdm.avoidExternalConnections=true</code>	<code>false</code>	Avoid connections to external urls, useful when downloading webdrivers from an artifact storage in an intranet
<code>browserVersionDetectionCommand(String)</code>	<code>wdm.browserVersionDetectionCommand</code>	<code>""</code> (automatic discovery using the commands database)	Custom browser version detection command (see example here)
<code>useLocalCommandsPropertiesFirst()</code>	<code>wdm.commandsPropertiesOnlineFirst=true</code>	<code>false</code> (the online commands database is used in the resolution algorithm)	Use local copy of the commands database
<code>commandsPropertiesUrl(URL)</code>	<code>wdm.commandsPropertiesUrl</code>	Raw version of the online commands database	Change commands database URL
<code>clearDriverCache()</code>	<code>wdm.clearDriverCache=true</code>	<code>false</code> (not cleaning driver cache)	Clean driver cache
<code>clearResolutionCache()</code>	<code>wdm.clearResolutionCache=true</code>	<code>false</code> (not cleaning resolution cache)	Clean resolution cache
<code>ttl(int)</code>	<code>wdm.ttl</code>	<code>86400</code> (i.e., 1 day)	TTL in seconds in which the resolved driver versions are valid in the resolution cache.

API method	Configuration key	Default value	Description
<code>ttlBrowsers(int)</code>	<code>wdm.ttlForBrowsers</code>	3600 (i.e., 1 hour)	TTL value in seconds in which the browser versions are valid in the resolution cache (also used for dockerized browsers).
<code>disableTracing()</code>	<code>wdm.tracing=false</code>	true (RemoteWebDriver tracing enabled by default)	Disable OpenTelemetry tracing for RemoteWebDriver

Table 2. Configuration capabilities for browsers in Docker

API method	Configuration key	Default value	Description
<code>dockerDaemonUrl(String)</code>	<code>wdm.dockerDaemonUrl</code>	""	URL of remote Docker daemon
<code>dockerTimezone(String)</code>	<code>wdm.dockerTimezone</code>	Etc/UTC	Timezone of the browser container
<code>dockerNetwork(String)</code>	<code>wdm.dockerNetwork</code>	bridge	Docker network name
<code>dockerLang(String)</code>	<code>wdm.dockerLang</code>	EN	Language of the browser container
<code>dockerShmSize(String)</code>	<code>wdm.dockerShmSize</code>	512m	Docker shared memory in bytes. Unit is optional and can be b (bytes), k (kilobytes), m (megabytes), or g (gigabytes)
<code>dockerTmpfsSize(String)</code>	<code>wdm.dockerTmpfsSize</code>	128m	Docker in-memory filesystem (tmpfs). Units follows the same approach than the shared memory
<code>dockerTmpfsMount(String)</code>	<code>wdm.dockerTmpfsMount</code>	/tmp	Mount point for in-memory filesystem
<code>dockerStopTimeoutSec(Integer)</code>	<code>wdm.dockerStopTimeoutSec</code>	5	Max time to kill a container (in seconds) after stopping it
<code>enableVnc()</code>	<code>wdm.dockerEnableVnc=true</code>	false	Enable desktop remote session for browsers in Docker
<code>viewOnly()</code>	<code>wdm.dockerViewOnly=true</code>	false	Run remote desktop session (noVNC) in view-only mode

API method	Configuration key	Default value	Description
<code>enableRecording()</code>	<code>wdm.dockerEnableRecording=true</code>	<code>false</code>	Enable the recordings of the browser session in Docker
<code>dockerRecordingFrameRate(int)</code>	<code>wdm.dockerRecordingFrameRate</code>	<code>15</code>	Frame rate for recordings
<code>dockerScreenResolution(String)</code>	<code>wdm.dockerScreenResolution</code>	<code>1920x1080x24</code>	Screen resolution of the browser desktop session in format <code><width>x<height>x<colors-depth></code>
<code>dockerRecordingOutput(String)</code> <code>dockerRecordingOutput(Path)</code>	<code>wdm.dockerRecordingOutput</code>	<code>.</code>	Path for the recording output. This value can be a folder or complete path (if it ends with <code>.mp4</code>)
<code>dockerRecordingPrefix(String)</code>	<code>wdm.dockerRecordingPrefix</code>	Browser name	Prefix to be appended to default filename (i.e., browser name plus <code>_</code> plus session id)
<code>dockerCustomImage(String)</code>	<code>wdm.dockerCustomImage</code>	<code>""</code>	Custom image to be used as browser in Docker
<code>dockerVolumes(String[])</code>	<code>wdm.dockerVolumes</code>	<code>""</code>	Docker volumes (single or array) using the format <code>"\local\path:\container\path"</code>
<code>dockerExtraHosts(String[])</code>	<code>wdm.dockerExtraHosts</code>	<code>""</code>	Docker Extra Hosts (single or array) using the format <code>"hostname:IP"</code> (<code>"host1:192.168.48.82, host2:192.168.48.16"</code>)
<code>dockerEnvVariables(String[])</code>	<code>wdm.dockerEnvVariables</code>	<code>""</code>	Environment variable for Docker containers`
<code>dockerDefaultArgs(String)</code>	<code>wdm.dockerDefaultArgs</code>	<code>--disable-gpu</code>	Default arguments to start Docker containers`

API method	Configuration key	Default value	Description
<code>dockerPrivateEndpoint(String)</code>	<code>wdm.dockerPrivateEndpoint</code>	<code>""</code>	Used to prefix pull images when you have a private registry with authentication i.e <code>docker-hub-remote.myprivate.com</code> will be prefixed to pull as <code>docker-hub-remote.myprivate.com/selenoid/vnc</code> and so on for any images used (video recorder, novnc etc.), docker login <code>docker-hub-remote.myprivate.com</code> is still required in order to get the auth credentials stored in the <code>.docker/config.json</code> , for MacOS users make sure to configure your engine to store the credentials in the <code>config.json</code> instead of keychain storage.
<code>dockerAvoidPulling()</code>	<code>wdm.dockerAvoidPulling</code>	<code>false</code>	Avoid pulling Docker images from Docker Hub

Table 3. Configuration capabilities for WebDriverManager Server

API method	Configuration key	Default value	Description
<code>serverPort(int)</code>	<code>wdm.serverPort</code>	<code>4444</code>	Port of WebDriverManager Server
<code>serverPath(String)</code>	<code>wdm.serverPath</code>	<code>/</code>	Path of WebDriverManager Server
<code>serverTimeoutSec(int)</code>	<code>wdm.serverTimeoutSec</code>	<code>60</code>	Timeout (in seconds) for WebDriverManager server

Chapter 7. Known Issues

7.1. HTTP response code 403

Some of the drivers (e.g., geckodriver or operadriver) are hosted on GitHub. When external clients (like WebDriverManager) make many consecutive requests to GitHub, and due to its traffic rate limit, it eventually responds with an HTTP 403 error (*forbidden*), as follows:

```
io.github.bonigarcia.wdm.config.WebDriverManagerException: Error HTTP 403 executing
https://api.github.com/repos/mozilla/geckodriver/releases
    at io.github.bonigarcia.wdm.online.HttpClient.execute(HttpClient.java:172)
    at
io.github.bonigarcia.wdm.WebDriverManager.openGitHubConnection(WebDriverManager.java:1
266)
    at
io.github.bonigarcia.wdm.WebDriverManager.getDriversFromGitHub(WebDriverManager.java:1
280)
    at
io.github.bonigarcia.wdm.managers.FirefoxDriverManager.getDriverUrls(FirefoxDriverMana
ger.java:95)
    at
io.github.bonigarcia.wdm.WebDriverManager.createUrlHandler(WebDriverManager.java:1111)
    at io.github.bonigarcia.wdm.WebDriverManager.download(WebDriverManager.java:959)
    at io.github.bonigarcia.wdm.WebDriverManager.manage(WebDriverManager.java:877)
    at io.github.bonigarcia.wdm.WebDriverManager.fallback(WebDriverManager.java:1106)
    at
io.github.bonigarcia.wdm.WebDriverManager.handleException(WebDriverManager.java:1083)
    at io.github.bonigarcia.wdm.WebDriverManager.manage(WebDriverManager.java:883)
    at io.github.bonigarcia.wdm.WebDriverManager.setup(WebDriverManager.java:328)
```

To avoid this problem, WebDriverManager can make authenticated requests using a [personal access token](#). See the [advanced configuration](#) section to discover how to set up this token in WebDriverManager.



As of WebDriverManager 5.3.0, this issue should not happen anymore (even without a GitHub token). To avoid the 403 error, an automated job in GitHub Actions mirrors the GitHub API responses twice a day. This mirror is used internally by WebDriverManager instead of querying the GitHub API.

7.2. Testing localhost in Docker

A typical case in web development is testing a web application deployed in the localhost. In this case, and when using browsers in Docker containers, we need to know that the address **localhost** inside a Docker container is not the host's address but the container address. To solve this problem, we can take different approaches. In Linux, we can use the gateway address for inter-container communication. This address is usually **172.17.0.1**, and can be discovered as follows:

```
$ ip addr show docker0
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN
group default
    link/ether 02:42:b4:83:10:c8 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 scope global docker0
        valid_lft forever preferred_lft forever
```

When the host is Mac OS or Windows, we can use the DNS name `host.docker.internal`, which will be resolved to the internal IP address used by the host.

7.3. Chrome 92-94 in Docker

An [issue](#) affected Chrome 92 to 94 and Docker. The problem can be solved by using the argument `--disable-gpu` in Chrome and Edge. This argument is used by WebDriverManager 5 to avoid the issue. Nevertheless, some situations are still impossible to fix (e.g., when using [WebDriverManager in Docker](#) as CLI or Server) for Chrome 92 to 94 in Docker.

7.4. Support for chromedriver 115+

The chromedriver team has stopped publishing the chromedriver releases and metadata using their traditional [chromedriver download repository](#) with chromedriver 114. This way, as of **chromedriver 115**, the chromedriver releases can only be discovered programmatically using the [Chrome for Testing \(CfT\) JSON endpoints](#).

This change is very relevant for WebDriverManager, since, as of Chrome 115 and 116, chromedriver cannot be managed automatically by WebDriverManager using the traditional way. Therefore, for older versions of WebDriverManager, this situation led to errors like the following:

```
io.github.bonigarcia.wdm.online.HttpClient: Error HTTP 404 executing
https://chromedriver.storage.googleapis.com/LATEST_RELEASE_116
```

```
org.openqa.selenium.SessionNotCreatedException: Could not start a new session.
Response code 500. Message: session not created: This version of ChromeDriver only
supports Chrome version 114
```

WebDriverManager 5.4+ implements the support for the CfT endpoints. Therefore, the solution to this problem is to bump WebDriverManager to the **latest version** (6.4.0 currently). Also, to ensure that the wrong version has not been cached in the **resolution cache**, you can refresh completely the cache folder (at least once) as follows:

```
WebDriverManager.chromedriver().clearDriverCache().setup();
```

7.5. Connectivity issues

WebDriverManager must request remote endpoints (like the Chrome for Testing (CfT) endpoints) and download drivers from online repositories. When this operation is done in a corporate environment with a proxy or firewall, it might lead to connectivity problems like the following:

```
Error HTTP 403 executing https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-testing/116.0.5845.96/win64/chromedriver-win64.zip
```

If this is your case, consider the following:

- Use the proxy capabilities of WebDriverManager to set up your proxy, e.g.:

```
WebDriverManager.chromedriver().proxy("myproxy:port").setup();
```

- Review your network setup to enable the remote requests and downloads required by WebDriverManager.

Chapter 8. Troubleshooting

For getting the WebDriverManager logs, you need to include a Logback configuration file (for example, like [this](#)) in your project classpath. Then, you need to set the level to **DEBUG**, or **TRACE** for finer details, e.g.:

```
<logger name="io.github.bonigarcia" level="TRACE" />
```

After that, when running a test, you should see something like this:

```
2023-08-25 13:10:11 [main] DEBUG i.g.bonigarcia.wdm.WebDriverManager.<init>(227) -
Using WebDriverManager {project-version}
2023-08-25 13:10:12 [main] TRACE
i.g.b.wdm.versions.VersionDetector.getVersionsInputStream(364) - Reading online
commands.properties to find out driver version
2023-08-25 13:10:12 [main] DEBUG
i.g.b.wdm.versions.VersionDetector.getBrowserVersionFromTheShell(253) - Detecting
chrome version using online commands.properties
2023-08-25 13:10:12 [main] DEBUG i.g.bonigarcia.wdm.versions.Shell.runAndWaitArray(65)
- Running command on the shell: [cmd.exe, /C, wmic, datafile, where,
name="%PROGRAMFILES:\=\%\Google\Chrome\Application\chrome.exe", get, Version,
/value]
2023-08-25 13:10:12 [main] DEBUG i.g.bonigarcia.wdm.versions.Shell.runAndWaitArray(69)
- Result: Version=116.0.5845.110
2023-08-25 13:10:12 [main] TRACE
i.g.b.wdm.versions.VersionDetector.getBrowserVersionUsingCommand(332) - Detected
browser version is 116.0.5845.110
2023-08-25 13:10:12 [main] INFO
i.g.bonigarcia.wdm.WebDriverManager.resolveDriverVersion(1280) - Using chromedriver
116.0.5845.96 (resolved driver for Chrome 116)
2023-08-25 13:10:12 [main] DEBUG
i.g.b.wdm.cache.ResolutionCache.putValueInResolutionCacheIfEmpty(119) - Storing
resolution chrome=116 in cache (valid until 14:10:12 25/08/2023 CEST)
2023-08-25 13:10:12 [main] DEBUG
i.g.b.wdm.cache.ResolutionCache.putValueInResolutionCacheIfEmpty(119) - Storing
resolution chrome116=116.0.5845.96 in cache (valid until 13:10:12 26/08/2023 CEST)
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.getDriverFromCache(83) -
Checking if chromedriver exists in cache
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.filterCacheBy(68) -
Filter cache by chromedriver -- input list
[C:\Users\boni\.cache\selenium\resolution.properties] -- output list []
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.filterCacheBy(68) -
Filter cache by 116.0.5845.96 -- input list [] -- output list []
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.filterCacheBy(68) -
Filter cache by WIN -- input list [] -- output list []
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.getDriverFromCache(105)
- Avoid filtering for architecture 64 with chromedriver in Windows
2023-08-25 13:10:12 [main] TRACE i.g.b.wdm.cache.CacheHandler.getDriverFromCache(119)
```

```

- chromedriver not found in cache
2023-08-25 13:10:12 [main] INFO i.g.bonigarcia.wdm.WebDriverManager.logSeekRepo(1631)
- Reading https://chromedriver.storage.googleapis.com/ to seek chromedriver
2023-08-25 13:10:12 [main] DEBUG
i.g.bonigarcia.wdm.WebDriverManager.createUrlHandler(1516) - Driver to be downloaded
chromedriver 116.0.5845.96
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.WebDriverManager.createUrlHandler(1525) - Driver URLs after
filtering for version: []
2023-08-25 13:10:12 [main] TRACE i.g.bonigarcia.wdm.online.UrlHandler.filterByOs(141)
- URLs before filtering by OS (WIN): []
2023-08-25 13:10:12 [main] TRACE i.g.bonigarcia.wdm.online.UrlHandler.filterByOs(145)
- URLs after filtering by OS (WIN): []
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.online.UrlHandler.filterByArch(151) - URLs before filtering by
architecture (64): []
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.online.UrlHandler.filterByArch(160) - URLs after filtering by
architecture (64): []
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.online.UrlHandler.filterByBeta(127) - URLs before filtering by beta
versions: []
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.online.UrlHandler.filterByBeta(134) - URLs after filtering by beta
versions: []
2023-08-25 13:10:12 [main] DEBUG i.g.bonigarcia.wdm.WebDriverManager.buildUrl(156) -
Using URL built from repository pattern:
https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-
testing/116.0.5845.96/win64/chromedriver-win64.zip
2023-08-25 13:10:12 [main] TRACE i.g.bonigarcia.wdm.online.Downloader.getTarget(123) -
Target file for URL https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-
testing/116.0.5845.96/win64/chromedriver-win64.zip driver version 116.0.5845.96 =
C:\Users\boni\.cache\selenium\chromedriver\win64\116.0.5845.96\chromedriver-win64.zip
2023-08-25 13:10:12 [main] INFO
i.g.bonigarcia.wdm.online.Downloader.downloadAndExtract(131) - Downloading
https://edgedl.me.gvt1.com/edgedl/chrome/chrome-for-
testing/116.0.5845.96/win64/chromedriver-win64.zip
2023-08-25 13:10:12 [main] TRACE
i.g.bonigarcia.wdm.online.Downloader.downloadAndExtract(137) - Target folder
C:\Users\boni\.cache\selenium\chromedriver\win64\116.0.5845.96 ... using temporal file
C:\Users\boni\AppData\Local\Temp\15824197595769158441\chromedriver-win64.zip
2023-08-25 13:10:13 [main] INFO i.g.bonigarcia.wdm.online.Downloader.extract(192) -
Extracting driver from compressed file chromedriver-win64.zip
2023-08-25 13:10:13 [main] TRACE i.g.bonigarcia.wdm.online.Downloader.unZip(220) -
Unzipping chromedriver-win64/LICENSE.chromedriver (size: 215994 KB, compressed size:
35448 KB)
2023-08-25 13:10:13 [main] TRACE i.g.bonigarcia.wdm.online.Downloader.unZip(220) -
Unzipping chromedriver-win64/chromedriver.exe (size: 14312960 KB, compressed size:
7094251 KB)
2023-08-25 13:10:13 [main] TRACE i.g.bonigarcia.wdm.online.Downloader.deleteFile(330)
- Deleting file C:\Users\boni\AppData\Local\Temp\15824197595769158441\chromedriver-

```

```
win64.zip
2023-08-25 13:10:13 [main] TRACE
i.g.bonigarcia.wdm.WebDriverManager.postDownload(1371) - Found driver in post-
download: C:\Users\boni\AppData\Local\Temp\15824197595769158441\chromedriver-
win64\chromedriver.exe
2023-08-25 13:10:13 [main] TRACE i.g.bonigarcia.wdm.online.Downloader.deleteFile(330)
- Deleting file C:\Users\boni\AppData\Local\Temp\15824197595769158441\chromedriver-
win64\LICENSE.chromedriver
2023-08-25 13:10:13 [main] TRACE
i.g.bonigarcia.wdm.online.Downloader.deleteFolder(340) - Deleting folder
C:\Users\boni\AppData\Local\Temp\15824197595769158441
2023-08-25 13:10:13 [main] TRACE
i.g.bonigarcia.wdm.online.Downloader.downloadAndExtract(163) - Driver after extraction
C:\Users\boni\cache\selenium\chromedriver\win64\116.0.5845.96\chromedriver.exe
2023-08-25 13:10:13 [main] INFO
i.g.bonigarcia.wdm.WebDriverManager.exportDriver(1333) - Exporting
webdriver.chrome.driver as
C:\Users\boni\cache\selenium\chromedriver\win64\116.0.5845.96\chromedriver.exe
```

For further info about logging with SLF4J and Logback you can see the following [tutorial](#).

Chapter 9. Community

There are two ways to try to get community support related to WebDriverManager. First, questions can be discussed in [StackOverflow](#), using the tag *webdrivermanager_java*. In addition, comments, suggestions, and bug-reporting should be made using the [GitHub issues](#). Finally, if you think WebDriverManager can be enhanced, consider contributing to the project through a [pull request](#).

Chapter 10. Support

[WebDriverManager](#) is part of [OpenCollective](#), an online funding platform for open and transparent communities. You can support the project by contributing as a backer (i.e., a personal [donation](#) or [recurring contribution](#)) or as a [sponsor](#) (i.e., a recurring contribution by a company).

Chapter 11. Further Documentation

There are other resources related to WebDriverManager and automated testing you can find helpful. For instance, the following books:

- García, Boni. [Hands-On Selenium WebDriver with Java](#). O'Reilly Media, Inc., 2022.
- García, Boni. [Mastering Software Testing with JUnit 5](#). Packt Publishing Ltd, 2017.

Or the following papers:

- García, Boni, et al., "[Test automation with selenium: A survey](#)." *Information and Software Technology* 194 (2026): 108077.
- Leotta, M., et al., "[A family of experiments to quantify the benefits of adopting WebDriverManager and Selenium-Jupiter](#)." *Information and Software Technology* 178 (2025): 107595.
- García, Boni, et al. "[Exploring Browser Automation: A Comparative Study of Selenium, Cypress, Puppeteer, and Playwright](#)." *International Conference on the Quality of Information and Communications Technology*. Springer Nature Switzerland, 2024.
- Leotta, M., et al. "[AI-generated test scripts for web E2E testing with ChatGPT and Copilot: a preliminary study](#)." *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 2024.
- García, Boni, et al. "[Enhancing Web Applications Observability through Instrumented Automated Browsers](#)." *Journal of Systems and Software* (2023): 111723.
- Leotta, M., et al. "[Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey](#)". In the 16th *IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2023, Dublin, Ireland. April 2023.
- Leotta, M., et al. "[An Empirical Study to Quantify the SetUp and Maintenance Benefits of Adopting WebDriverManager](#)." *15th International Conference on the Quality of Information and Communications Technology (QUATIC) 2022*, Talavera de la Reina, Spain. September 2022.
- García, Boni, et al. "[Selenium-Jupiter: A JUnit 5 extension for Selenium WebDriver](#)." *Journal of Systems and Software* (2022): 111298.
- García, Boni, et al. "[Automated driver management for Selenium WebDriver](#)." *Empirical Software Engineering* 26.5 (2021): 1-51.
- García, Boni, et al. "[A survey of the Selenium ecosystem](#)." *Electronics* 9.7 (2020): 1067.
- García, Boni, et al. "[Assessment of QoE for video and audio in WebRTC applications using full-reference models](#)." *Electronics* 9.3 (2020): 462.
- García, Boni, et al. "[Practical evaluation of VMAF perceptual video quality for WebRTC applications](#)." *Electronics* 8.8 (2019): 854.
- García, Boni, et al. "[WebRTC testing: challenges and practical solutions](#)." *IEEE Communications Standards Magazine* 1.2 (2017): 36-42.
- García, Boni, and Juan Carlos Dueñas. "[Web browsing automation for applications quality control](#)." *Journal of web engineering* (2015): 474-502.

Chapter 12. About

WebDriverManager (Copyright © 2015-2026) is an open-source project created and maintained by [Boni García \(@boni_gg\)](#), licensed under the terms of [Apache 2.0 License](#). This documentation (also available in [PDF](#)) is released under the terms of [CC BY-NC-SA 2.0](#).